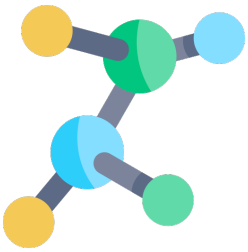


Molecular Analysis Interface

Multi-database Molecular Analyzer

Author :
Noé Gabillaud

Academic Supervisor :
François Bondu



May/June 2026

Technical Documentation

Contents

1 Introduction **2**

2 Quick Start **4**
2.1 Requirements 4
2.2 Installing Dependencies 4
2.3 Launching the Interface 4
2.3.1 Results Through the HTML Interface 4
2.3.2 Results via the Python interpreter 7

3 Project Architecture **8**
3.1 File Organization 8
3.2 Data Flow 8
3.3 Role of Parsers 9

4 Module Guide **10**
4.1 Software Orchestration: The `app.py` Module 10
4.2 The Parsers: Data Extraction 11
4.2.1 PubChem Parser 11
4.2.2 Astroquery Parser: Spectral Resonance Points 12
4.2.3 Thermo-Chemical Parser: Thermodynamic Data 13
4.2.4 NIST Parser: Extraction via Scraping 13
4.2.5 mVOC Parser: Browser Automation 14
4.2.6 Sander Parser: Henry's Constants 15
4.3 Calculation Tools and Utilities 16
4.4 Visualization and Chart Generation 17
4.5 Interface Management: HTML Templates 17

5 HTML Interface Features **19**
5.1 Light/Dark Mode 19
5.2 Export/Print 19
5.3 Exit 19
5.4 Links and Copy 19
5.5 H Codes 19
5.6 CSV Export 20

Chapter 1

Introduction

This document provides the technical documentation for the Molecular Analysis Interface. Its purpose is to present the architecture and functionality of the tool, thereby facilitating its adoption and use by users.

The BacTera experimental setup at the FOTON Institute is designed to measure the volatile metabolites produced by microbes, particularly those found in agricultural soils, using millimeter-wave and submillimeter-wave spectroscopy, in collaboration with the ECOBIO Laboratory. In the long term, this instrument may contribute to the assessment of the biological functioning of soils. As part of this project, access to information about the studied molecules is required. However, the relevant data are distributed across multiple databases. The purpose of the Molecular Analysis Interface is therefore to consolidate information from these various databases into a single, unified platform.

The required data are as follows:

- General information about the molecule's identity
- Molar mass
- LogP value
- Boiling point
- Vapor pressure
- Dipole moment
- Enthalpies of vaporization and fusion
- Henry's law constant
- Safety information
- The spectrum over the selected frequency range and resolution
- Peak frequency and Doppler linewidth

The databases used are:

- PubChem
- NIST
- mVOC
- Sander
- CDMS, JPL, and HITRAN via Astroquery
- Thermo-Chemical

Chapter 2

Quick Start

2.1 Requirements

Before getting started, make sure your working environment is properly configured. The project requires the following elements:

- **Python** installed on your machine.
- **Spyder** (or any other Python IDE).
- The libraries listed in the `requirements.txt` file.

2.2 Installing Dependencies

The required libraries can be installed by running the `installation.py` script in a Python interpreter. Alternatively, to automatically install all necessary dependencies, open a terminal in the project directory and execute the following command:

```
pip install -r requirements.txt
```

Listing 2.1: Package Installation

2.3 Launching the Interface

Once the dependencies have been installed, the application can be launched in three different ways: through the `.exe` file named "Windows Interface Launcher" (available only on the Windows operating system), through the `launcher.py` script (for other operating systems) to display the results in an HTML interface (recommended method), or through the `main.py` script to display the results directly in the Python interpreter.

2.3.1 Results Through the HTML Interface

Using the `.exe` File

To launch the interface:

1. Open the `.exe` file named "Windows Interface Launcher".
2. The system will automatically launch `app.py`.

Using the `launcher.py` File

To launch the interface:

1. Open the `launcher.py` file in your editor.
2. Run the script.
3. The system will automatically launch `app.py`.

After launching, your default web browser should automatically open at `http://127.0.0.1:5000/`. If the page loads correctly, your installation is complete and functional. You can then search for the desired molecule (name, formula or PubChem CID) using the search engine, along with the required parameters.

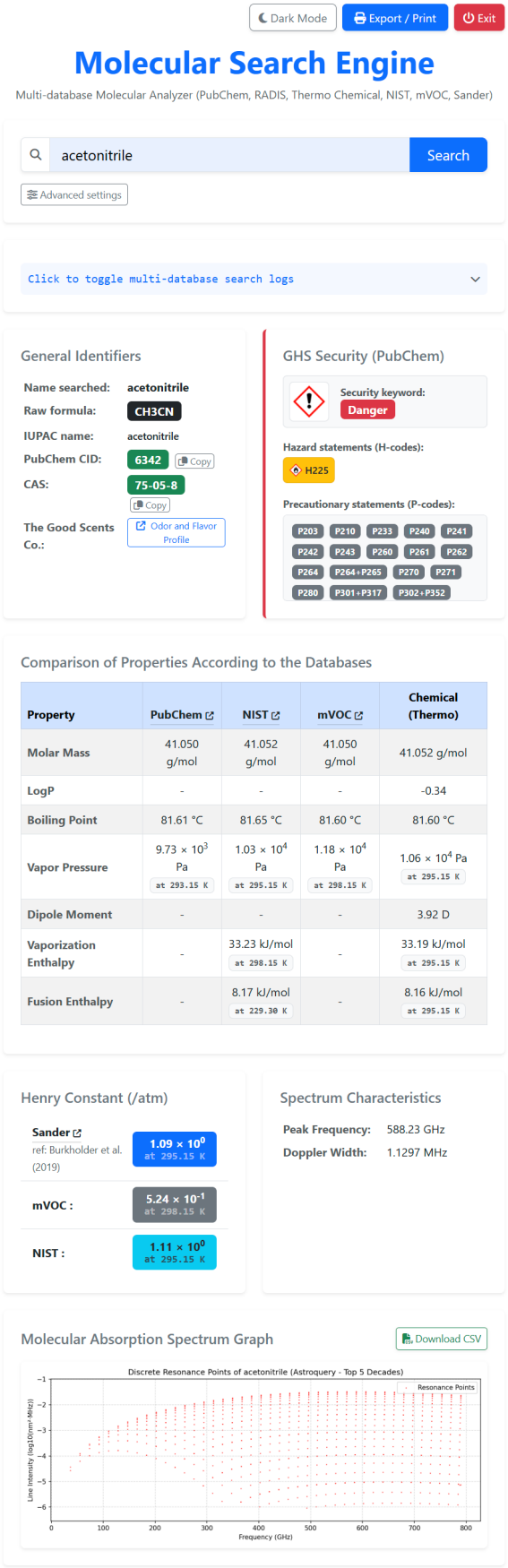


Figure 2.1: Example: interface displayed for acetonitrile

2.3.2 Results via the Python interpreter

To retrieve the data:

1. Open the `main.py` file in your editor.
2. Enter the name or formula of the desired molecule on line 13.
3. Optionally adjust the associated search parameters.
4. Run the script.

```
[PubChem] OK : CAS=75-05-8 CID=6342
[Astroquery]: CH3CN via CDMS (Discrete)
[Chemical]: OK
[NIST]: OK
[mVOC]: OK
[Sander]: OK

=== Formula ===
C2H3N

=== IUPAC Name ===
acetonitrile

=== SMILES ===
CC#N

=== Molar Mass ===
PubChem = 41.05 g/mol, Chemical = 41.052 g/mol, NIST = 41.052 g/mol, mVOC = 41.05
     g/mol

=== logP ===
PubChem = None, Chemical = -0.34, mVOC = 0.0

=== Boiling Point ===
PubChem = 81.61 °C, Chemical = 81.6 °C NIST = 81.65 °C, mVOC = 81.6 °C

=== Vapor Pressure ===
PubChem = 11839.03 Pa at 298.15 K, Chemical = 12146.63 Pa at 298.15 K, NIST =
    11795.09 Pa at 298.15 K, mVOC = 11839.03 Pa at 298.15 K

=== Dipole Moment ===
PubChem = None D, Chemical = 3.92 D, NIST = None D

=== Vaporization Enthalpy ===
PubChem = None kJ/mol at None K, Chemical = 33.05 kJ/mol NIST = 33.23 kJ/mol at
    298.15 K

=== Fusion Enthalpy ===
PubChem = None kJ/mol at None K, Chemical = 8.16 kJ/mol NIST = 8.17 kJ/mol at
    229.3 K

=== Henry Constant ===
HenryConstant(Sander = 0.95 /atm at 298.15 K, mVOC = 0.52 /atm at 298.15 K, NIST
    = 0.97 /atm at 298.15 K

=== Security ===
Signal = ['Danger'], H = ['H225'], P = ['P203', 'P210', 'P233', 'P240', 'P241', '
    P242', 'P243', 'P260', 'P261', 'P262', 'P264', 'P264+P265', 'P270', 'P271', '
    P280', 'P301+P317', 'P302+P352', 'P303+P361+P353', 'P304+P340', 'P305+P351+
    P338', 'P305+P354+P338', 'P308+P316', 'P316', 'P317', 'P318', 'P319', 'P321',
     'P330', 'P337+P317', 'P361+P364', 'P362+P364', 'P370+P378', 'P403+P235', '
    P405', 'P501']

=== Spectrum ===
Peak frequency = 588.46 GHz, Doppler width = 1.136 MHz
```

Listing 2.2: Example: text obtained for acetonitrile

Note: The spectrum is also plotted.

Chapter 3

Project Architecture

This chapter presents the overall structure of the source code as well as the application's functionality.

3.1 File Organization

Each file in the project has a specific role:

- `Windows Interface Launcher.exe`: main entry point of the program for Windows. It checks the Python environment before starting the server.
- `Interface_Launcher_via_Python.py`: universal main entry point of the program. It also checks the Python environment before starting the server.
- `app.py`: Flask-based main controller. It manages interactions between the user and the different parsers.
- `main.py`: console version of the program, allowing the parsers to be executed without a web interface.
- `solver.py`: coordinates the different parsers and centralizes the retrieved data for searches via `main.py`.
- `/templates`: contains HTML files defining the structure of the web pages.
- `/static`: contains the resources used by the interface (images, styles, etc.).

3.2 Data Flow

The application's operation is carried out in several steps:

1. The user enters a molecule via the web interface or directly in the Python program.
2. The controller (`app.py` or `solver.py`) forwards the request to the different parsers.
3. Each parser queries its database, extracts the relevant information, and then normalizes it.
4. The retrieved data is then displayed either in the web interface or in the Python console.

3.3 Role of Parsers

The parsers ensure communication between the application and external databases such as PubChem, CDMS, JPL, HITRAN, Chemical, NIST, mVOC, or Sander.

Each parser has three main functions:

- **Request**: send a query to a database via an API or web scraping.
- **Normalization**: convert the retrieved data into a common format in order to standardize units and structures.
- **Error handling**: prevent program interruptions when data is missing or when a database does not respond.

This organization makes it possible to isolate each data source. Thus, if a database changes its behavior, only the corresponding parser needs to be modified without impacting the rest of the project.

Chapter 4

Module Guide

This chapter describes the internal organization of the various components used for the extraction, processing, and display of molecular data.

4.1 Software Orchestration: The `app.py` Module

The `app.py` file acts as the central hub of the application. Unlike modules specialized in data extraction, it coordinates the entire processing workflow, from receiving the user request to displaying the results on the HTML page.

Request Lifecycle Management

The module uses the `Flask` framework to define the application routes. Its role is to chain together the various operations required to process a molecule:

1. **Reception and validation:** retrieval of user input (molecule name or formula) then initialization of the `molecule` object.
2. **Service calling:** execution of the different parsers (PubChem, NIST, mVOC, etc.) based on available identifiers (CAS, CID).
3. **Physical calculations:** launching normalization tools and extrapolation functions, particularly for vapor pressure using Antoine parameters or Henry's law.
4. **Visual rendering:** passing the enriched `molecule` object to the `Jinja2` template engine to generate the results page.

Robustness and Exception Handling

To ensure that an error from a single data source does not crash the entire application, `app.py` integrates global exception handling:

- **Error isolation:** each service call is encapsulated in a `try/except` block. When a data point is unavailable, the value is left empty without preventing the display of other properties.
- **Operation tracking:** the module generates logs to track the different execution steps and facilitate debugging (progress bar on the HTML page).

Object-Oriented Architecture

The module relies on a unified data structure based on the `molecule` class. All data sources — APIs, web scraping, or local calculations — feed into the same standardized attributes.

This organization simplifies the addition of new sources: it is sufficient to develop a new parser and then integrate it into the execution sequence of `app.py`.

4.2 The Parsers: Data Extraction

Each file in the `/parsers` directory is dedicated to a single data source.

4.2.1 PubChem Parser

The `pubchem_parser.py` module retrieves molecular information from the PubChem database. Since the data returned by the API is deeply nested, the parser uses an exploration method tailored to the JSON structure provided by PubChem.

Recursive Property Extraction

The `PUG View` API returns a JSON file whose organization varies depending on the molecule. The `extract_pubchem_property` function recursively traverses this structure:

- searching for sections containing the `TOCHeading` key corresponding to the requested property;
- extracting the values found within `StringWithMarkup`.

This method allows the retrieval of properties such as boiling point, vapor pressure, or safety data even when their position in the JSON changes from one molecule to another.

Cleaning and Standardization

Since the extracted data is heterogeneous, the parser applies several treatments before it is used:

1. **Normalization via regular expressions:** cleaning functions convert textual scientific notations into numbers usable by Python.
2. **Selection of experimental conditions:** for temperature-dependent properties, the `get_closest_to_temperature` method automatically selects the experimental value closest to the requested temperature.

GHS Data Extraction

The parser also retrieves chemical safety information via the `get_ghs` function:

- detecting signal words such as “Danger” or “Warning” ;
- extracting H and P codes using regular expressions.

The obtained data is then reused in the interface to display the corresponding pictograms and warnings.

4.2.2 Astroquery Parser: Spectral Resonance Points

The `astroquery_parser.py` module is responsible for retrieving discrete spectral lines and resonance points for a given molecule. To do this, it remotely queries reference spectroscopic catalogs via `astroquery` services, namely `CDMS`, `JPL`, and `HITRAN`.

Molecule Resolution

One of the main challenges consists in matching the name or formula provided by the user with the strict conventions and identifiers required by the remote servers.

The parser performs several verification and conversion steps for this purpose:

- normalization of chemical formulas with the `normalize_formula` function, which decomposes and counts the number of atoms in the molecule ;
- automatic retrieval of the empirical formula via the PubChem API (`pubchempy`) if the initial identifier entered is a common name ;
- matching with the target molecule dictionary in order to associate the correct numerical key or search *tag*.

Querying and Fallback Strategy

When a search is initiated within the defined frequency band, the parser applies a sequential querying strategy to ensure data acquisition, even if some services are unavailable:

1. attempt to retrieve discrete lines from the `CDMS` (*Cologne Database for Molecular Spectroscopy*) database ;
2. in case of failure or absence of lines, automatic fallback to the `JPL` (*Jet Propulsion Laboratory*) catalog ;
3. as a last resort, querying the online `HITRAN` tables.

Each network request is encapsulated in a timeout manager (`run_with_timeout`) limiting execution to 25 seconds, thereby protecting the application against excessive latencies or remote server failures.

Physical Processing and Doppler Calculation

Once the set of discrete lines is successfully retrieved, the parser applies the following physical transformations:

- conversion of spectral frequencies into gigahertz (GHz) and application of a thermal correction to the line intensities (`log_ints`) if the requested temperature deviates from the catalog's reference temperature ;
- identification of the main resonance line (the peak of maximum intensity) within the frequency band ;

- calculation of the Doppler width at half-maximum with `calculate_doppler_width` at this main peak. This calculation depends on the temperature and molar mass of the molecule (extracted from NIST or PubChem databases) according to the formula:

$$\Delta\nu = \frac{\nu_{\text{pic}}}{c}\sqrt{\frac{8k_B T \ln 2}{m}}$$

with m the mass of a molecule and ν_{pic} the frequency of maximum resonance.

This organization allows for the rapid isolation of critical frequencies of a molecule in a stable, lightweight manner, without relying on large local databases.

Laboratory rumor: Some claim that by clicking three times in a row on "physical and spectral parameters", the code awakens a hidden mechanism and a completely different, continuous spectral simulation takes shape on the screen...

4.2.3 Thermo-Chemical Parser: Thermodynamic Data

The `chemical_parser.py` module completes thermodynamic properties using the `thermo` library.

Search via CAS Number

The parser uses the `CAS` number obtained from PubChem to query the internal database of the `thermo` library. This method limits ambiguities related to chemical names.

Property Processing

The retrieved properties are directly mapped to the attributes of the `molecule` object:

- Conversion of vaporization and fusion enthalpies into kJ/mol ;
- Conversion of temperatures from Kelvin to degrees Celsius ;
- Calculation of the saturated vapor pressure at the user-defined temperature.

Error Handling

When the molecule is not present in the `thermo` library or the CAS number is invalid, the parser catches the exception without interrupting the rest of the processing workflow.

4.2.4 NIST Parser: Extraction via Scraping

The `nist_parser.py` module complements thermodynamic data by querying the NIST Chemistry WebBook. As this database does not offer a direct API, information is retrieved by analyzing the HTML code.

Navigation and Extraction

Extraction follows several steps:

- Sending requests with parameters tailored to the searched data ;
- Analyzing the HTML with `BeautifulSoup` ;
- Reconstructing property tables from `tr` and `td` tags ;
- Extracting numerical values using regular expressions ;
- **Filtering and data selection:** to guarantee data relevance, the parser prioritizes isolating valid coefficients within the temperature range of the study or, failing that, extracts the data corresponding to the closest available temperature in the NIST tables.

Physical Calculations

The parser can also calculate certain properties:

1. **Antoine Equation:** calculation of the saturated vapor pressure when the A, B, and C coefficients are available using: $P = 10^{A-\frac{B}{T+C}}$ with T the temperature ;
2. **Henry's Constant:** extrapolation of the constant to the user's temperature from the coefficients provided by NIST using: $H = H_0 e^{\texttt{dlnHs/dT}(\frac{1}{T}-\frac{1}{298.15K})}$

Handling the Unexpected

Like any scraping system, this module heavily depends on the site's structure. If a page or table is missing, values are left as `None` so as not to interrupt the overall processing.

4.2.5 mVOC Parser: Browser Automation

The `mvoc_parser.py` module extracts physicochemical and ecological data from the mVOC database. Since the website relies on a dynamic interface, the parser automates navigation to submit forms and capture the generated content.

Isolation and Automation with Playwright

To interact with the interface, the parser relies on the `Playwright` library in headless mode (without a graphical interface). The software architecture was designed to guarantee cross-platform compatibility and maximum stability:

- **Windows Event Loop Management:** On the `win32` platform, the parser forces the use of `WindowsSelectorEventLoopPolicy` to properly support the underlying asynchronous processes.
- **Isolation by Dedicated Thread:** Unlike heavy process-based execution, the navigation engine is encapsulated in an independent thread (`_thread_worker`). This isolation is essential to avoid major conflicts related to the `asyncio` event loop of the main application or development environments such as Spyder.

The automation workflow of the retrieval function (`_fetch_mvoc_html`) performs the following actions: opening the Chromium browser, navigating to the search page, entering the PubChem CID (`compound_id`), submitting the form, and securely intercepting the final HTML code transferred via a shared dictionary (`container`).

Advanced Data Extraction

Once the HTML code is retrieved, the `fill_from_mvoc` function orchestrates data extraction using `BeautifulSoup` combined with targeted regular expressions:

1. **Global characteristics table:** The parser iterates through the `tr` tags to retrieve key data such as *LogP*, the IUPAC name, the SMILES code, the molar mass (*Molweight*), as well as the biological origin (*in_ organism*).

2. **Henry's constant:** The code searches for the keyword "henry" and applies specific `regex` patterns capable of capturing the value, whether written in scientific notation ($a \times 10^{-b}$) or as a raw textual estimate.

3. **Boiling points and vapor pressure:** The parser scans all HTML tables. It extracts the boiling point in degrees Celsius and isolates the saturated vapor pressure data by dynamically identifying the original unit (`mmHg`, `Pa`, `atm`, `kPa`) as well as the associated temperature, converted directly into Kelvin.

Normalization and Integration

The extracted raw data undergoes a cleaning and uniformization process before integration. The module uses core utility functions (`core.tools`):

- `preprocess` is applied to numerical strings to standardize textual formats ;
- `convert_value` gelle les changements d'unités physiques (par exemple, la conversion de la pression de vapeur de `mmHg` vers l'unité standard de l'application, le Pascal `Pa`).

At the end of the process, the cleaned values are directly injected into the dedicated sub-attributes of the `molecule` object (`mol.molar_mass.mvoc`, `mol.logp.mvoc`, `mol.boilingtemp.mvoc`, etc.), ensuring full traceability of the `mvoc` data source within the system.

4.2.6 Sander Parser: Henry's Constants

The `sander_parser.py` module is dedicated to extracting Henry's constants from the Sander database.

Search via CAS Number

The parser uses the CAS number as the primary identifier to avoid ambiguities between closely related molecules.

Data Selection

Since multiple entries can be available for the same molecule, the parser applies a selection logic:

- priority given to data containing the `dlnHs/dT` coefficient and calculation of Henry's constant H at temperature T by: $H = H_0 e^{\texttt{dlnHs/dT}(\frac{1}{T} - \frac{1}{298.15K})}$;
- use of a reference value when this coefficient is unavailable.

Cleaning and Conversion

The retrieved data is cleaned and then converted:

1. removal of special characters present in the HTML ;
2. calculation of Henry's constant at the desired temperature if the `dlnHs/dT` coefficient is available ;
3. conversion of Henry's constants into homogeneous units (atm^{-1}).

This module is primarily used for calculations related to the solubility of gaseous compounds.

4.3 Calculation Tools and Utilities

The `tools.py` file groups together the utility functions used by the various parsers.

- **Unit conversions:** pressure, temperature, energy, enthalpy, etc.
- **Data validation:** verification of values before their display.

The module primarily serves to uniformize data coming from different sources.

Data Preprocessing

The `preprocess` function cleans character strings before conversion:

- replacement of Unicode superscripts ;
- standardization of scientific notation ;
- correction of typographical characters.

Unit Conversion

The module uses the `pint` library to perform conversions:

1. automatic detection of units present in the text ;
2. conversion to common units ;
3. handling lines containing multiple value/unit pairs.

Physical Functions

Certain functions directly implement physical relationships:

- extrapolation of Henry's constant with `henry_at_T` ;
- formatting of numbers with `clean_float`.

Thanks to these tools, parsers can focus on data extraction while `tools.py` ensures their homogenization.

4.4 Visualization and Chart Generation

The graphic generation module transforms spectral data into actionable representations within the web interface.

Rendering Architecture

The system relies on `matplotlib`, configured with the *Agg* backend. This configuration allows generating figures on the server side without a local graphical interface.

Generation Function

The `plot_absorption_spectrum` function follows several steps:

1. configuration of axes and chart style ;
2. generation of the image in a memory buffer ;
3. Base64 encoding of the image ;
4. systematic closure of the figure with `plt.close()`.

This method avoids creating temporary files and reduces disk access.

Error Handling

If no spectral data is available, the function simply returns an empty value to avoid a server-side error.

4.5 Interface Management: HTML Templates

The `/templates` folder contains the HTML pages used by Flask.

- **Jinja2:** template engine used to dynamically inject data into HTML pages ;
- **Bootstrap or custom CSS:** layout management and adaptation to different screen sizes.

Layout of the Results Page

The `results.html` page groups information into several sections:

- **Identifiers block:** name, empirical formula, PubChem CID, SMILES representation, as well as a link to The Good Scents Company website;
- **Spectrum block:** display resonance points ;
- **Comparative table:** grouping of properties from different sources ;
- **Other information:** display of Henry's constants and safety information.

Visualization and Safety

The template integrates several dynamic elements:

1. Display of GHS pictograms from H and P codes ;
2. Direct integration of the spectrum as a Base64 encoded image.

User Experience

The interface applies several presentation choices:

- use of scientific notation for certain values ;
- adding links to external databases ;
- displaying colored badges based on data availability.

The combination allows presenting data from various sources in a homogeneous and user-friendly interface.

Chapter 5

HTML Interface Features

This chapter describes the different functionalities of the HTML page.

5.1 Light/Dark Mode

A `Light/Dark Mode` button has been implemented to change the page color theme. Font colors and table styling are adjusted accordingly. Spectral colors are also inverted to ensure consistency with the overall display.

5.2 Export/Print

The `Export/Print` button allows exporting the page results as a PDF file or printing them directly. It is recommended to switch to `Light Mode` before exporting or printing the page.

5.3 Exit

The Exit button allows the program to be properly closed, without using the `.exe` window or the Python interpreter.

5.4 Links and Copy

By clicking on database names in the results table, the molecule's page on the corresponding database is opened in a new tab. There are also buttons to copy the molecule's CID and CAS identifiers. The CID is required to search for the molecule on mVOC.

5.5 H Codes

Each H code is associated with a pictogram and an explanation. The meaning of an H code can be displayed by hovering the mouse over it.

5.6 CSV Export

The button located at the top right of the spectrum allows downloading a `.csv` file containing the spectral data points over the selected frequency range and resolution used during the molecule search. There may be a delay between clicking the button and the download, due to the potentially large amount of data being generated.